

CSY3025 Artificial Intelligence Techniques

Decision Trees

https://github.com/ageron/handson-ml2/blob/master/06_decision_trees.ipynb

Decision Tree

- Like SVMs, Decision Trees are versatile Machine Learning algorithms that can perform both classification and regression tasks, and even multioutput tasks.
- Decision Trees are also the fundamental components of Random Forests, which are among the most powerful Machine Learning algorithms available today.

Training and Visualizing a Decision Tree

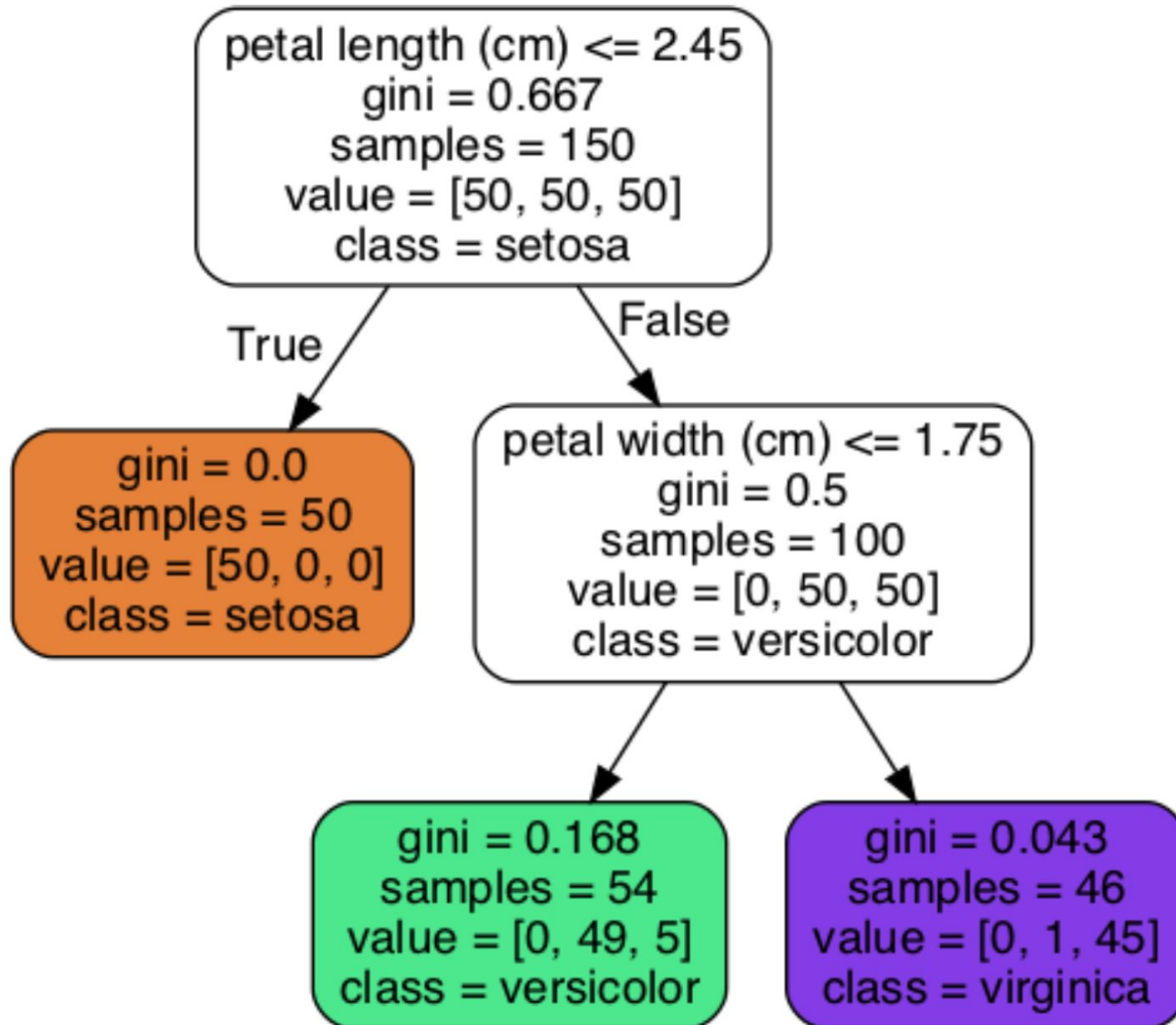
- train a DecisionTreeClassifier on the iris dataset

```
from sklearn.datasets import load_iris  
from sklearn.tree import DecisionTreeClassifier
```

```
iris = load_iris()  
X = iris.data[:, 2:] # petal length and width  
y = iris.target
```

```
tree_clf = DecisionTreeClassifier(max_depth=2)  
tree_clf.fit(X, y)
```

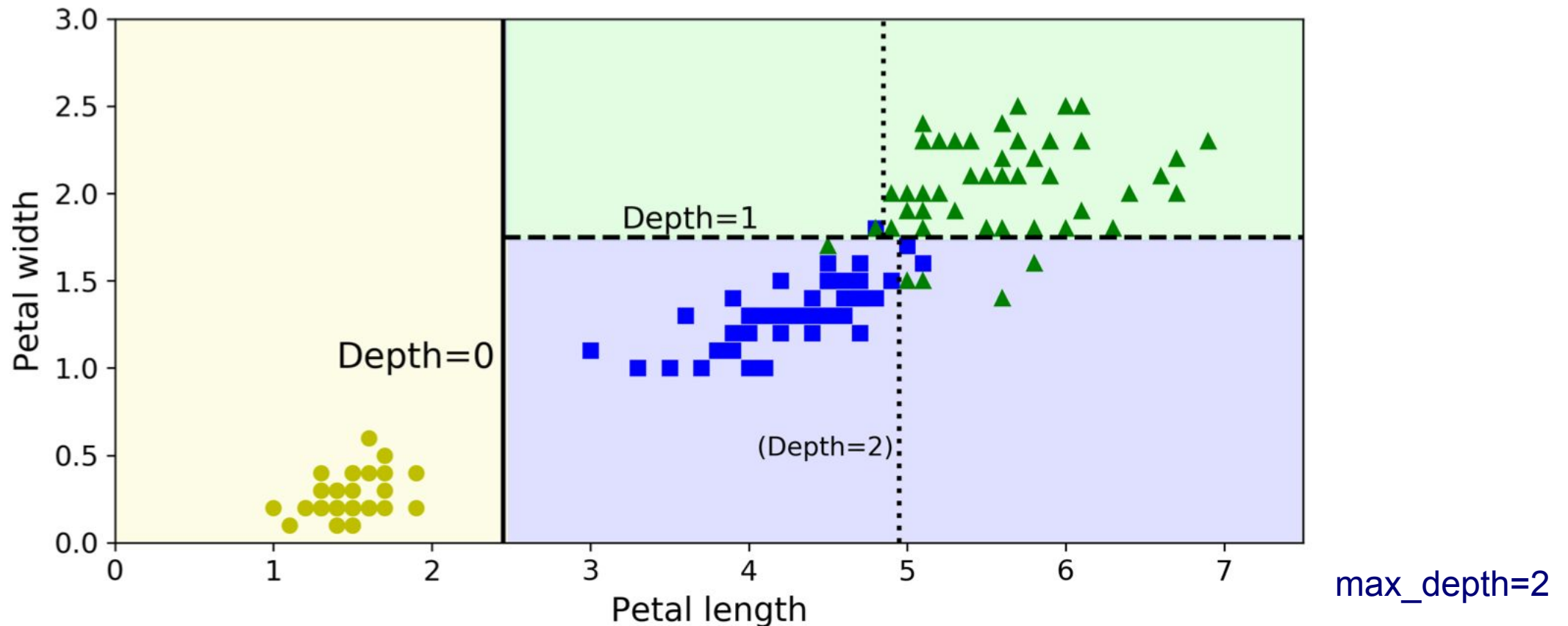
Training and Visualizing a Decision Tree



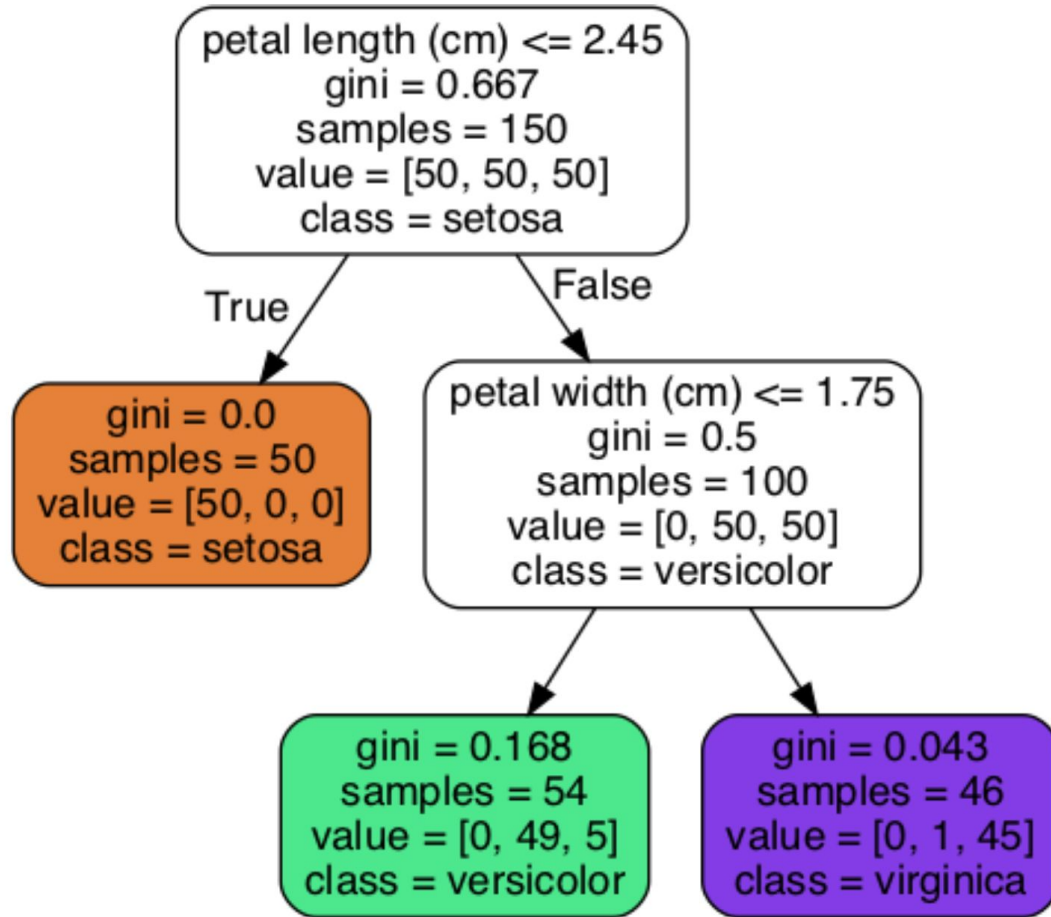
- A node's *samples* attribute counts how many training instances it applies to.
- A node's *value* attribute tells you how many training instances of each class this node applies to.
- a node's *gini* attribute measures its impurity.
- Scikit-Learn uses the CART algorithm, which produces only binary trees: nonleaf nodes always have two children.

Decision Tree decision boundaries and interpretability

- As you can see Decision Trees are fairly intuitive and their decisions are easy to interpret.
 - Such models are often called **white box models**. In contrast, neural networks are generally considered **black box models**.



Estimating Class Probabilities



- A Decision Tree can also estimate the probability that an instance belongs to a particular class k
 - first it traverses the tree to find the leaf node for this instance, and then it returns the ratio of training instances of class k in this node.
- For example, suppose you have found a flower whose petals are 5 cm long and 1.5 cm wide.
 - the Decision Tree should output the following probabilities: 0% for Iris-Setosa (0/54), 90.7% for Iris-Versicolor (49/54), and 9.3% for Iris-Virginica (5/54).

The CART Training Algorithm

- Scikit-Learn uses the *Classification And Regression Tree (CART)* algorithm to train Decision.
- The idea is really quite simple: the algorithm first splits the training set in two subsets using a single feature k and a threshold t_k (e.g., “petal length ≤ 2.45 cm”) recursively.
- It searches for the pair (k, t_k) that produces the purest subsets (weighted by their size).

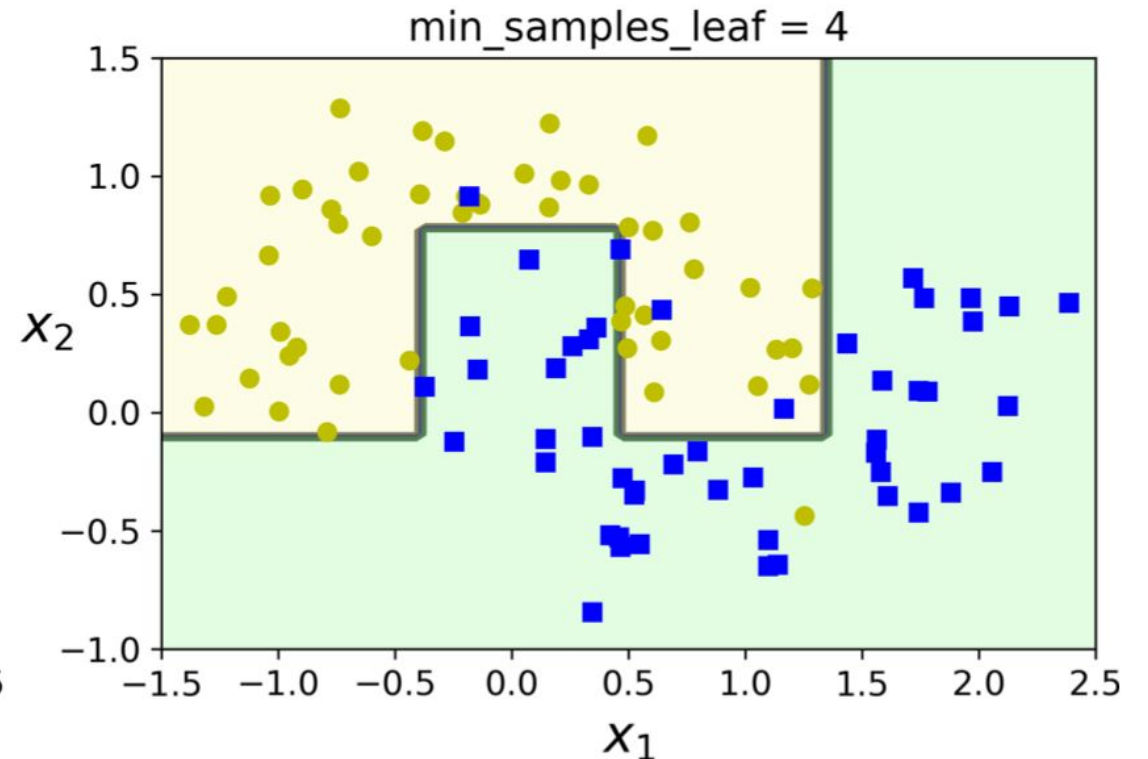
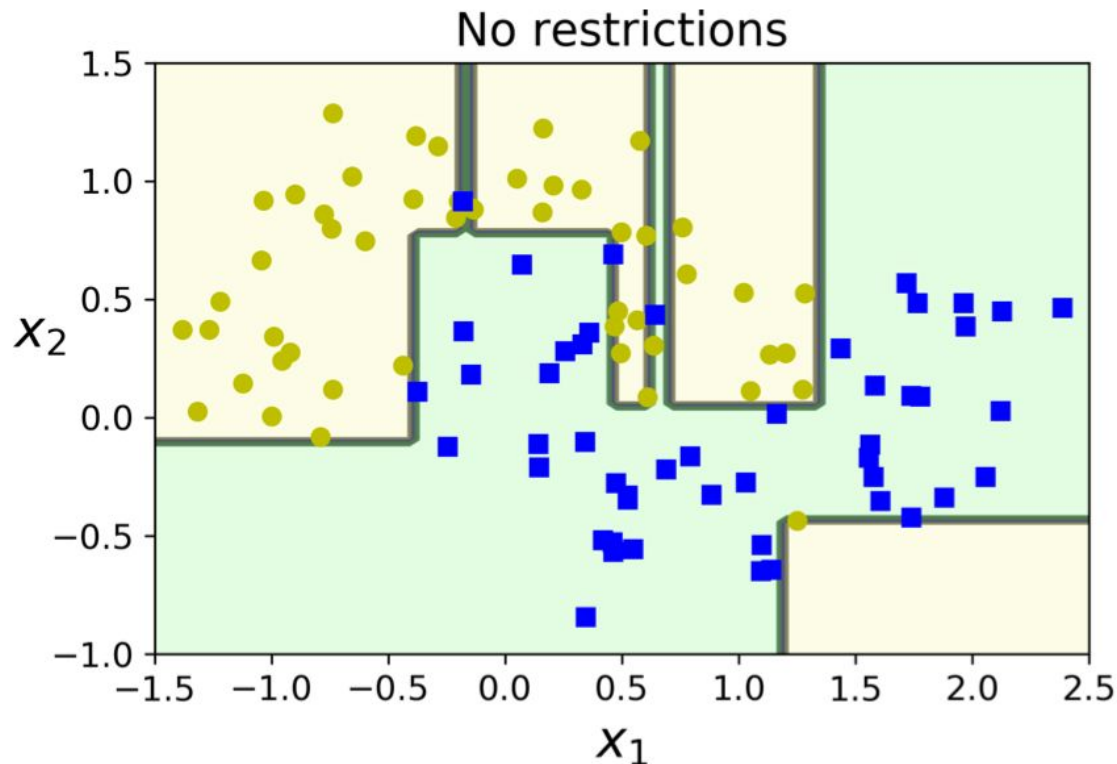
CART cost function for classification

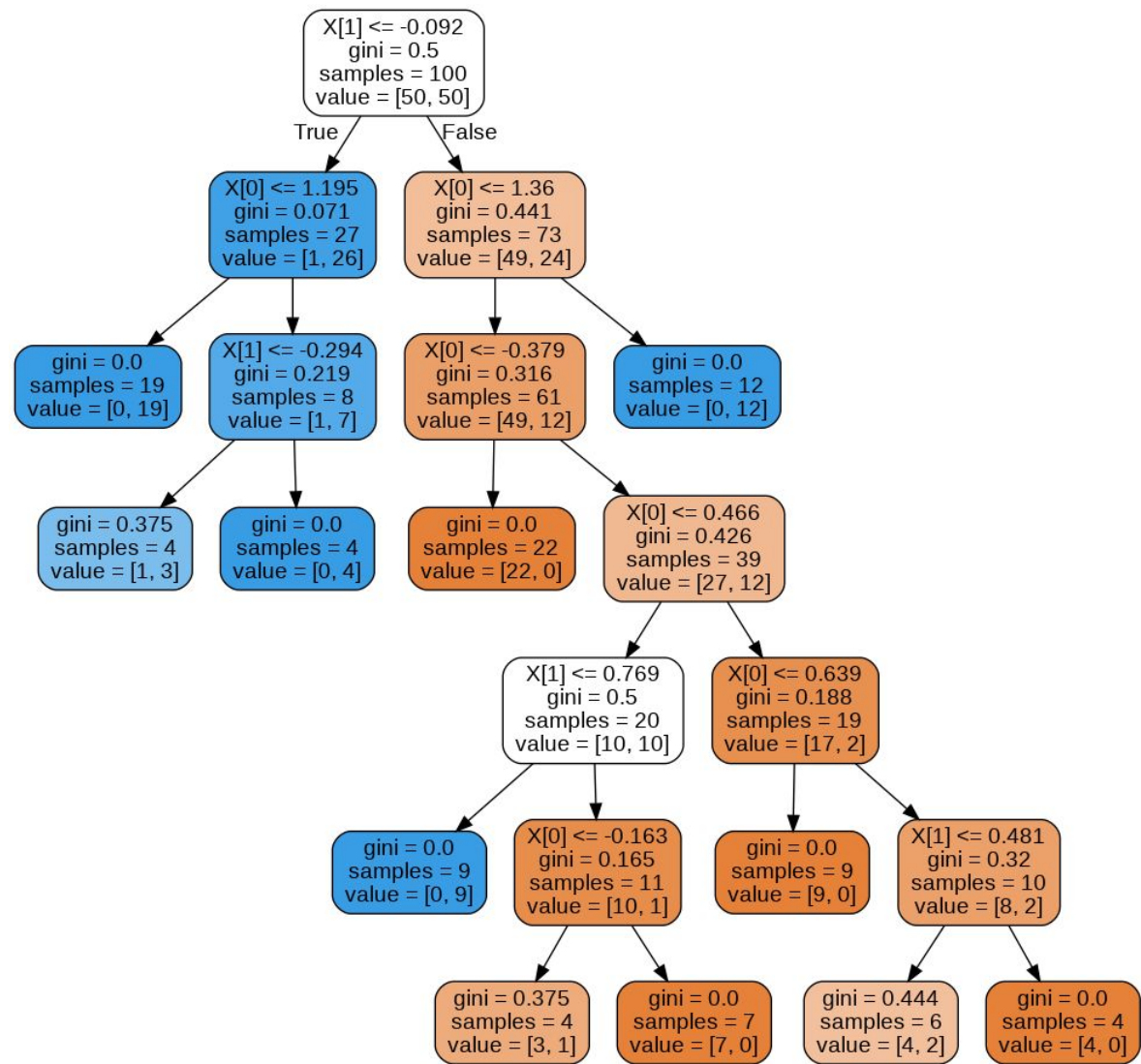
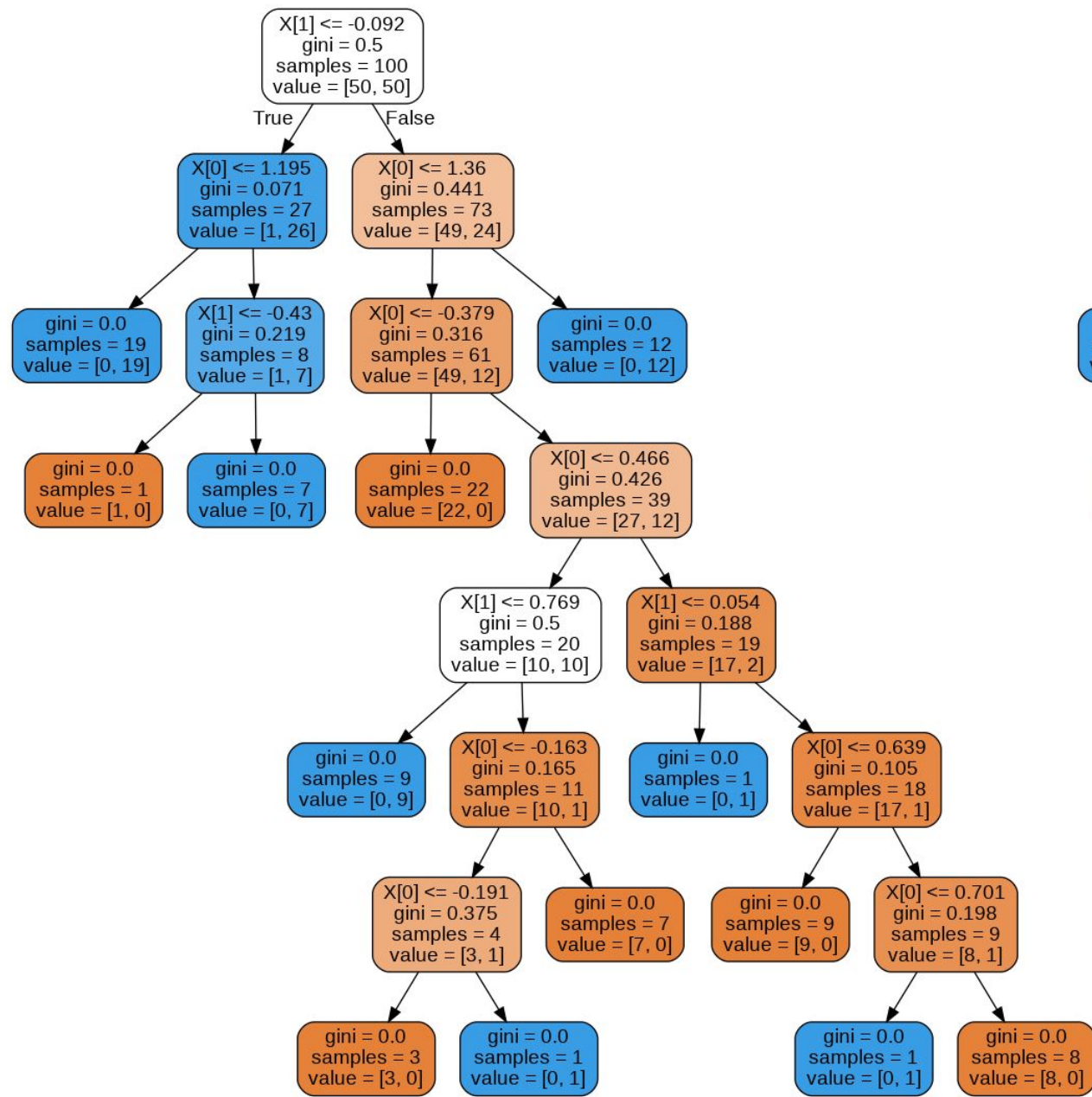
$$J(k, t_k) = \frac{m_{\text{left}}}{m} G_{\text{left}} + \frac{m_{\text{right}}}{m} G_{\text{right}}$$

where $\begin{cases} G_{\text{left/right}} & \text{measures the impurity of the left/right subset,} \\ m_{\text{left/right}} & \text{is the number of instances in the left/right subset.} \end{cases}$

Regularization

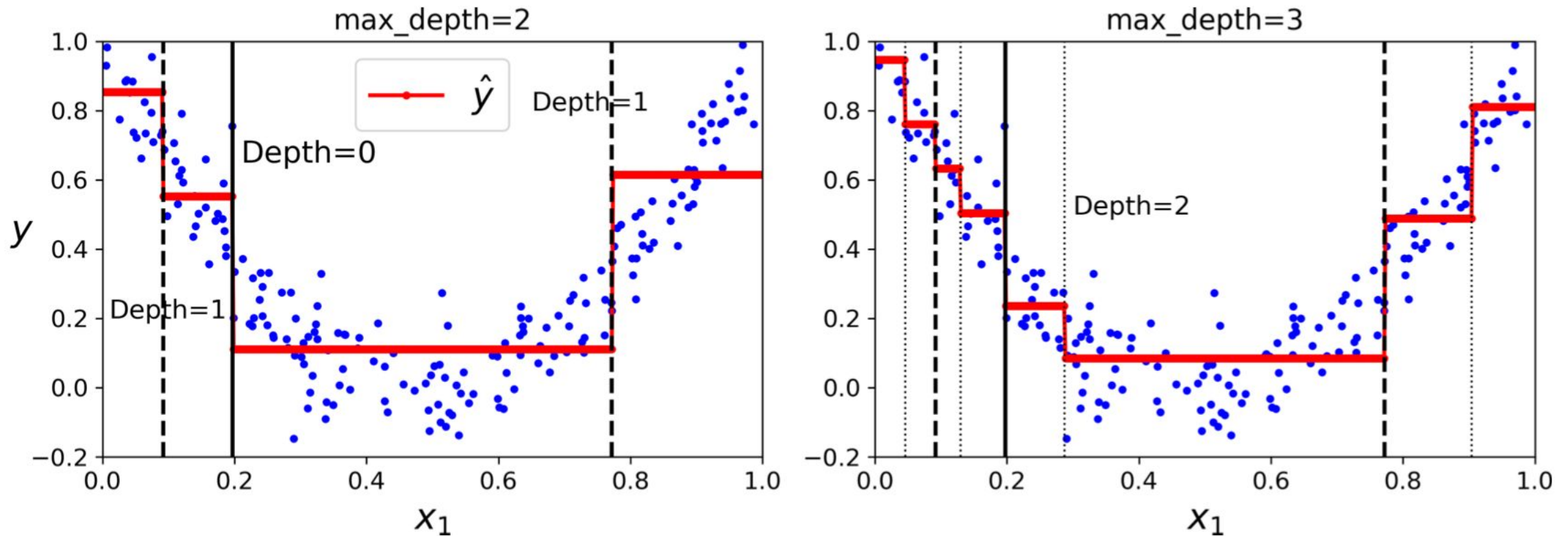
- To avoid overfitting the training data, you need to restrict the Decision Tree's freedom during training such as restricting the maximum depth of the Decision Tree.
 - The DecisionTreeClassifier class has a few other options: *min_samples_split* (the minimum number of samples a node must have before it can be split), *min_samples_leaf* (the minimum number of samples a leaf node must have), *max_leaf_nodes* (maximum number of leaf nodes), and *max_features* (maximum number of features that are evaluated for splitting at each node).





Regression

- Decision Trees are also capable of performing regression tasks. Let's build a regression tree using Scikit-Learn's `DecisionTreeRegressor` class.



Instability

- First, as you may have noticed, Decision Trees love orthogonal decision boundaries (all splits are perpendicular to an axis), which makes them sensitive to training set rotation.
- On the left, a Decision Tree can split it easily, while on the right, after the dataset is rotated by 45° , the decision boundary looks unnecessarily convoluted and won't generalise well..

